



VibeCodingGitRepoLinks...
PDF



Lab_4_VibeCoding_Stude...
PDF

my topic is the 10th one in the list , guide me step wise

Worked for 10s >

Yep bro. **Your topic is #10 → Breakout.** The repo assigned to you is

SETAPESU26/10_breakout . [VibeCodingGitRepoLinks](#)

I checked the actual README for your assigned repo, so we can do this **exactly according to the lab requirements**, step by step.

The lab specifically wants you to: clone/fork → run the original → record 10 sec before → use an LLM to fix the bug → add the 3 features → record 10 sec after → push to your own repo. [Lab_4_VibeCoding_Student_handou...](#)

Step 0 — Understand exactly what you're building

Your Breakout game already has:

- Paddle controlled by Left/Right arrows
- Ball bouncing off walls/paddle
- Bricks
- Bricks-left counter

But it has **1 deliberate bug + 3 features** to implement.

Your 4 tasks

Task	What you need to do
1. Fix bug	Bricks currently don't disappear when their hits reach 0
2. Lives	Add 3 lives + reset ball + game over + restart
3. Brick types	Normal, Strong, Unbreakable
4. Combo scoring	Score + multiplier that increases with consecutive brick hits

So don't randomly modify the game. **These four things are your assignment.**

Step 1 — Get the repo

The assigned repository is:

10_breakout GitHub repository

You **cannot push directly** to this repo because the repository permissions show you don't have push access. So you need your **own GitHub repository**.

The lab says to clone/fork the assigned repo and then push your changes to your personal repo; it explicitly says **don't raise a PR to SETAPESU26**.

Lab_4_VibeCoding_Student_handou...

Easiest method

Go to the repo → **Fork** → fork it into your GitHub account.

Name it something like:

10_breakout

or

Lab4_10_breakout

Step 2 — Clone YOUR fork

Once you've forked it, copy the URL of **your fork**.

In PowerShell:

```
cd Desktop
git clone YOUR_REPO_URL
cd 10_breakout
```

Then:

PowerShell

if you're using VS Code.

Step 3 — Inspect the project BEFORE touching anything

Your folder should roughly look like:

```
10_breakout/  
|  
├─ main.py  
├─ requirements.txt  
├─ README.md  
|  
└─ game/  
    ├─ game_engine.py  
    ├─ paddle.py  
    ├─ ball.py  
    ├─ brick.py  
    ├─ collision.py  
    └─ renderer.py
```

That's the structure specified in your repo's README.

Do not change anything yet.

Step 4 — Install dependencies

Open PowerShell inside the project:

PowerShell

You need Python 3.10+ according to the README.

Then:

PowerShell

Then:

PowerShell

Step 5 — Understand the original bug

This is VERY important because you need to demonstrate the bug in your first video.

Play the game.

Move the paddle with:

← →

Hit the bricks.

According to the README, the bug is:

The brick's `hits_remaining` gets reduced, but the brick is never actually removed.

So your original game should show something like:

```
Ball
  ↓
[BRICK]
  ↓ hit
[BRICK] ← still there
```

even though its hits have been used.

Step 6 — Record the BEFORE video

Do this before making ANY code changes.

The lab specifically requires a **10-second video before changes**, showing the broken behavior. Lab_4_VibeCoding_Student_handou...

Record approximately:

```
0 sec → start game
2 sec → move paddle
4 sec → ball hits brick
6 sec → show brick remains
8 sec → hit it again
10 sec → stop
```

Make sure the bug is visible.

Important

Don't record yourself coding.

Record the **gameplay**.

Save it as:

```
before.mp4
```

Step 7 — NOW use ChatGPT/Claude for Task 1

This is actually the core of the "Vibe Coding" assignment.

The README specifically tells you to use an LLM as a debugging/pair-programming partner.

Don't just dump:

```
fix my game
```

Instead, use a structured prompt.

Prompt 1 — Bug fix

Give your LLM this:

```
I am working on a Python/Pygame Breakout game.
```

```
I need to fix ONLY Task 1 from the README first.
```

```
Current bug:
```

```
In game/game_engine.py, when the ball hits a brick, brick.hits_remaining is decreased,
```

```
Please:
```

1. Inspect the existing code carefully.
2. Explain exactly where the bug is.
3. Make the smallest possible change needed to remove a brick once its hits_remaining
4. Do not implement lives, brick types, scoring, combos, or any other features yet.
5. Preserve the existing architecture and controls.
6. Give me the exact code changes and explain them.



Then let the LLM analyze the actual code.

Step 8 — Don't blindly accept the AI code

This part matters because your assignment is specifically about **Vibe Coding + critical code review**.

After it gives you a solution:

1. Understand what changed.
2. Apply it.
3. Run:

PowerShell

4. Test the brick.

You want:

```
hits_remaining = 1
```

```
↓ ball hits
```

```
hits_remaining = 0
```

```
↓
```

```
brick removed
```

Step 9 — Task 2: Lives

Once Task 1 works, move to Task 2.

Your README explicitly requires:

- 3 lives
- Lose a life when ball falls below paddle
- Reset ball
- Display remaining lives
- Game over when lives = 0
- Restart mechanism

Use this prompt:

```
Task 1 is now working.
```

```
Now implement ONLY Task 2 from the README.
```

Requirements:

- Add a 3-life system.
- When the ball falls below the paddle, decrease lives by 1.
- Reset the ball to its starting position so the player gets another attempt.
- Display the remaining lives on screen.
- When all 3 lives are lost, show a clear Game Over screen.
- Provide a way to restart the game.
- Preserve the existing brick collision behavior.
- Do not implement brick types or combo scoring yet.

Before giving the solution, inspect the existing architecture and explain which files/

Then test:

Start

↓

3 lives

Miss ball

↓

2 lives

↓

ball resets

Miss

↓

1 life

Miss

↓

Game Over

Restart

↓

3 lives

Step 10 — Task 3: Brick types

Now implement the three required types.

Your README defines them as:

Normal

1 hit → destroyed

Strong

multiple hits → destroyed

Unbreakable

any number of hits → never destroyed

They also need visually distinct appearances.

Prompt:

Task 1 and Task 2 are complete and tested.

Now implement ONLY Task 3 from the README.

Add three brick types:

1. Normal:
 - destroyed after one hit
2. Strong:
 - requires multiple hits before destruction
3. Unbreakable:
 - never destroyed regardless of how many times it is hit

Requirements:

- Make the three types visually distinct.
- Ensure collision behavior is correct for each type.
- Integrate this cleanly with the existing Brick class and collision/game-engine logic
- Do not implement combo scoring yet.
- Do not break the existing lives/game-over system.
- Keep the code readable and avoid unnecessary architectural changes.

First explain the design you intend to use, then provide the code changes.



Step 11 — Task 4: Combo scoring

Finally implement scoring.

Your README says:

```
Destroy brick
↓
score increases
↓
destroy another brick
↓
combo multiplier increases
↓
higher score
```

Missing the ball resets the combo.

Prompt:

```
Tasks 1, 2 and 3 are complete and tested.
```

```
Now implement ONLY Task 4 from the README.
```

```
Add:
```

- A score system.
- A combo multiplier.
- Destroying bricks consecutively without missing the ball should increase the combo m
- Each subsequent brick destruction should give more points according to the multiplie
- Missing the ball should reset the combo multiplier.
- Display the current score.
- Display the current multiplier.

```
Requirements:
```

- Preserve the existing 3-life system.
- Preserve all three brick types.
- Unbreakable bricks must not incorrectly increase the score/combo.
- Keep the implementation compatible with the existing architecture.
- Avoid unnecessary rewrites.

```
First inspect the existing code and explain where score/combo state should live, then
```



Step 12 — Test EVERYTHING

Before recording your final video, test this checklist.

✅ Bug fix

```
Normal brick
↓
hit
```

↓

brick disappears

✓ Lives

3 → 2 → 1 → 0

✓ Ball reset

After losing a life:

ball disappears

↓

ball resets

↓

new attempt

✓ Game over

0 lives

↓

GAME OVER

✓ Restart

GAME OVER

↓

restart

↓

3 lives

✓ Normal brick

1 hit → destroyed

✓ Strong brick

hit

↓

still exists

hit

↓

still exists / damaged

```
final hit  
↓  
destroyed
```

✓ Unbreakable

```
hit  
hit  
hit  
hit  
↓  
still exists
```

✓ Score

```
destroy brick  
↓  
score increases
```

✓ Combo

```
brick → combo increases  
brick → combo increases  
brick → combo increases
```

✓ Miss

```
combo  
↓  
miss ball  
↓  
combo resets
```

Step 13 — Record AFTER video

The lab requires another 10-second gameplay video after all changes.

Lab_4_VibeCoding_Student_handou...

Don't try to demonstrate literally everything in 10 seconds.

Make it show as many important features as possible:

```
0-2 sec → game + lives + score visible  
2-4 sec → hit bricks  
4-6 sec → show score/combo  
6-8 sec → show different brick behavior  
8-10 sec → gameplay continuing
```

If possible, make the HUD clearly visible:

```
Score: 150  
Lives: 3  
Combo: x3
```

Save:

```
after.mp4
```

Step 14 — Push your code

Check:

```
PowerShell
```

Then:

```
git add .  
git commit -m "Complete Breakout Lab 4 tasks"  
git push origin main
```

Your changes should now be in **your fork**, not the SETAPESU26 repository.

The assignment explicitly says not to raise a PR to the main SETAPESU26 repo.

Lab_4_VibeCoding_Student_handou...

Step 15 — Prepare your Lab-4 submission

The official lab deliverables are:

```
Lab-4/  
|  
├─ before.mp4  
├─ after.mp4  
├─ updated-code/  
└─ chat-history.pdf
```

The handout says the Lab-4 folder should contain the before/after videos, updated code, and exported Chat/LLM history. Lab_4_VibeCoding_Student_handou...

The repo's own README describes the submission as the before video, after video, and complete Chat/LLM history.

Your exact workflow

So don't do everything at once. We'll do it sequentially:

```
STEP 1  
Fork repo  
  ↓  
STEP 2  
Clone repo  
  ↓  
STEP 3  
Install requirements  
  ↓  
STEP 4  
Run original game  
  ↓  
STEP 5  
Record BEFORE video  
  ↓  
STEP 6  
Task 1 — Fix brick bug  
  ↓  
TEST  
  ↓  
STEP 7  
Task 2 — Lives + Game Over  
  ↓  
TEST  
  ↓  
STEP 8  
Task 3 — 3 Brick Types  
  ↓  
TEST  
  ↓  
STEP 9
```

Task 4 – Score + Combo



TEST EVERYTHING



STEP 10

Record AFTER video



STEP 11

Push to YOUR GitHub



STEP 12

Lab-4 submission

Start with Step 1. Fork `SETAPESU26/10_breakout` into your own GitHub account. Once you've cloned it, **send me the output of `dir` (or a screenshot of the project folder)** and we'll do the original-game setup together before touching any code.

 Sources




```
C:\Users\mdaga>git clone https://github.com/aarush-170/Lab4_10_breakout.git
Cloning into 'Lab4_10_breakout'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (12/12), done.
remote: Total 15 (delta 0), reused 15 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (15/15), 5.46 KiB | 136.00 KiB/s, done.

C:\Users\mdaga>cd Lab4_10_breakout

C:\Users\mdaga\Lab4_10_breakout>dir
Volume in drive C is OS
Volume Serial Number is 2664-28F7

Directory of C:\Users\mdaga\Lab4_10_breakout

30-09-2026 08:27 <DIR>      .
30-09-2026 08:27 <DIR>      ..
30-09-2026 08:27          34 .gitignore
30-09-2026 08:27 <DIR>      breakout
30-09-2026 08:27          4,029 README.md
                2 File(s)          4,063 bytes
                3 Dir(s)  70,220,808,192 bytes free

C:\Users\mdaga\Lab4_10_breakout>
```

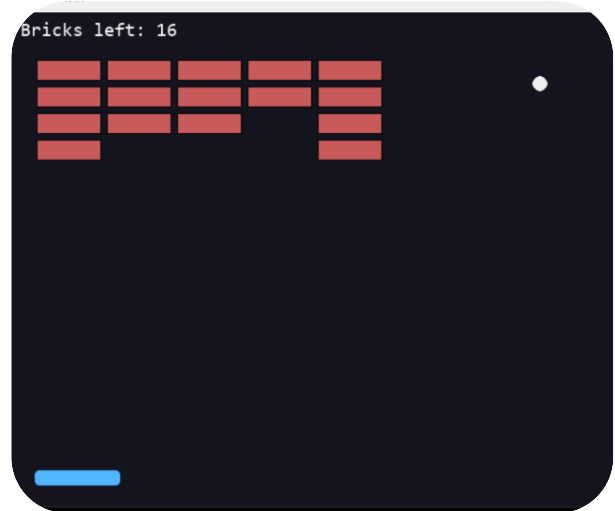


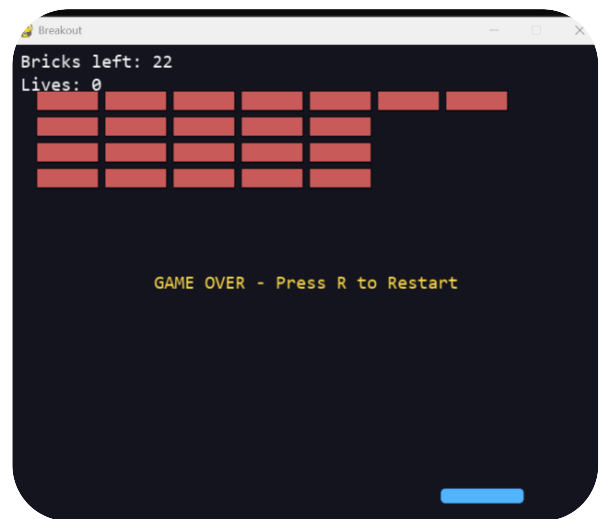

do i just record this ?



a7559005-9f2f-45a0-9d9...

File






```
C:\Users\mdaga\Lab4_10_breakout\breakout>git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   game/game_engine.py

no changes added to commit (use "git add" and/or "git commit -a")

C:\Users\mdaga\Lab4_10_breakout\breakout>git log --oneline --all
1382000 (HEAD -> main, origin/main, origin/HEAD) Initial Breakout starter

C:\Users\mdaga\Lab4_10_breakout\breakout>
```



```
C:\Users\mdaga\Lab4_10_breakout\breakout>git log --oneline --all
1382000 (HEAD -> main, origin/main, origin/HEAD) Initial Breakout starter

C:\Users\mdaga\Lab4_10_breakout\breakout>git add game/game_engine.py

C:\Users\mdaga\Lab4_10_breakout\breakout>git commit -m "Complete Tasks 1 and 2: brick removal and lives"
[main 97a482b] Complete Tasks 1 and 2: brick removal and lives
1 file changed, 98 insertions(+), 12 deletions(-)

C:\Users\mdaga\Lab4_10_breakout\breakout>git push origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.27 KiB | 434.00 KiB/s, done.
Total 5 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/aarush-170/Lab4_10_breakout.git
1382000..97a482b main -> main

C:\Users\mdaga\Lab4_10_breakout\breakout>
```

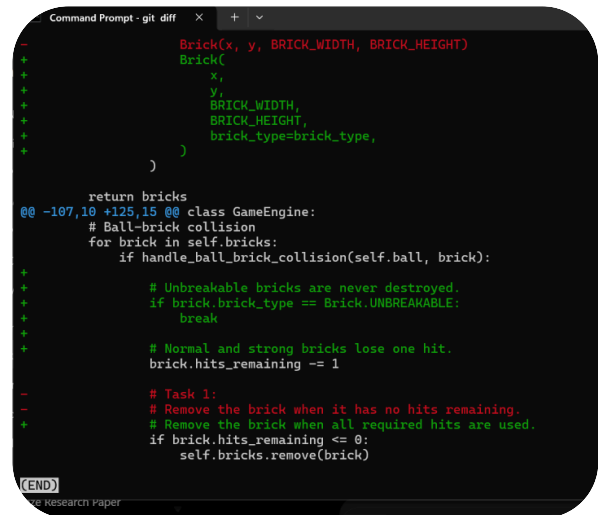
it also says dont raise pr to the main repo are we following that ?



```
--git a/breakout/game/brick.py b/breakout/game/brick.py
index e887ab4..3dcd332 100644
--- a/breakout/game/brick.py
+++ b/breakout/game/brick.py
@@ -1,18 +1,53 @@
 """
 -Brick: a single destructible block.
 +Brick: a single block in the Breakout game.
 +
 +Brick types:
 +- Normal: destroyed after 1 hit
 +- Strong: requires multiple hits
 +- Unbreakable: never destroyed
 """

import pygame

class Brick:
-    def __init__(self, x, y, width, height, hits_remaining=1, color=(200, 90, 90)):
+    NORMAL = "normal"
+    STRONG = "strong"
+    UNBREAKABLE = "unbreakable"
+
+    def __init__(
+        self,
+        x,
+        y,
+        width,
```

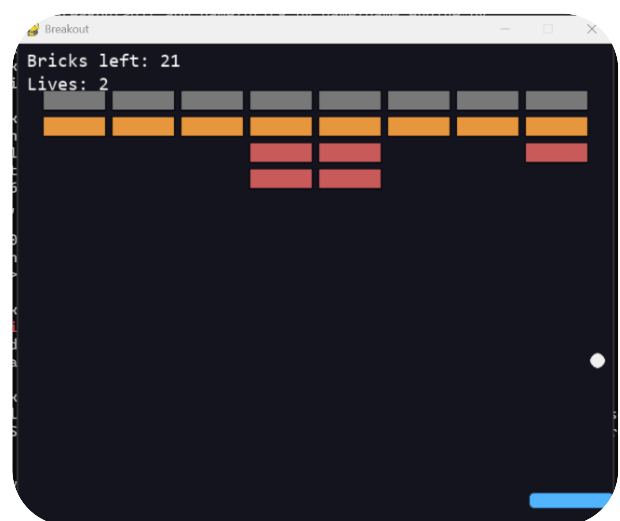



```
Command Prompt - git diff
-      Brick(x, y, BRICK_WIDTH, BRICK_HEIGHT)
+      Brick(
+          x,
+          y,
+          BRICK_WIDTH,
+          BRICK_HEIGHT,
+          brick_type=brick_type,
+      )
-
+    return bricks
@@ -107,10 +125,15 @@ class GameEngine:
     # Ball-brick collision
     for brick in self.bricks:
         if handle_ball_brick_collision(self.ball, brick):
+
+            # Unbreakable bricks are never destroyed.
+            if brick.brick_type == Brick.UNBREAKABLE:
+                break
+
+            # Normal and strong bricks lose one hit.
+            brick.hits_remaining -= 1
-
+            # Task 1:
+            # Remove the brick when it has no hits remaining.
+            # Remove the brick when all required hits are used.
+            if brick.hits_remaining <= 0:
+                self.bricks.remove(brick)
-
+    (END)
```

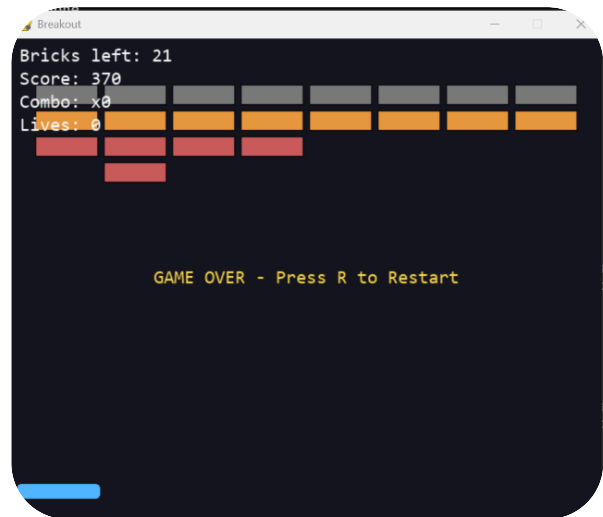
how do i exit this ?

```
C:\Users\mdaga\Lab4_10_breakout\breakout>git log --oneline --all
99c736b (HEAD -> main, origin/main, origin/HEAD) Complete Task 3: add brick types
97a482b Complete Tasks 1 and 2: brick removal and lives
8382000 Initial Breakout starter

C:\Users\mdaga\Lab4_10_breakout\breakout>
```

score not showing up



Yes — **you can record this final version.** 

For the recording, I'd show the functionality clearly rather than just letting the game run:

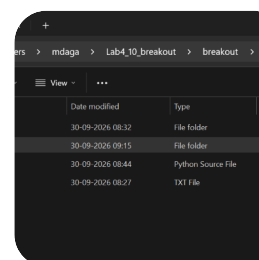
1. Start the game → show:

- Bricks left

- Score: 0
 - Combo: x0
 - Lives: 3
2. Move the paddle and hit a few **normal bricks** → show the score increasing.
 3. Hit a **strong (orange) brick** multiple times → demonstrate that it takes multiple hits.
 4. Hit an **unbreakable (gray) brick** → show that it remains.
 5. Let the ball fall → show:
 - Lives decreasing
 - Combo resetting to x0
 6. Lose all lives → show:
 - Lives: 0
 - GAME OVER - Press R to Restart
 7. Press R → show the game restarting with:
 - Score: 0
 - Combo: x0
 - Lives: 3
 - Full brick layout

That gives you a pretty solid demonstration of **Tasks 1–4 in one recording**.

One important thing: if the assignment specifically requires separate recordings/screenshots for each task, follow that requirement instead. Otherwise, this final gameplay recording is a good overall demo.



your Lab repo under Lab-4 folder,
before and after)
Code
ory exported as a doc/pdf

can i put the viedos like this and asll i need to put this chat history

Yes — based on the deliverables screenshot, you're on the right track.

Your current structure:

```
Lab4_10_breakout/  
└─ breakout/  
    │ game/  
    │ videos/  
    │ │ before.mp4  
    │ │ └─ after.mp4  
    │ main.py  
    └─ requirements.txt
```

is fine if `breakout` is the Lab-4 folder you're submitting.

The three deliverables are:

1. Videos — before and after

Put both recordings inside your `videos` folder:

```
videos/  
└─ before.mp4  
└─ after.mp4
```

The **before** should show the starter version, and **after** should show your completed Tasks 1–4.

2. Updated Code

You already have this in the repo:

```
game/  
main.py  
requirements.txt
```

And you've pushed the completed changes to **your fork**.

3. Chat history exported as DOC/PDF

Yes, you need to export this ChatGPT conversation as a **DOC or PDF** and put that file in the Lab-4 submission folder.

For example:

```
breakout/  
└─ game/  
└─ videos/  
    │ before.mp4  
    │ └─ after.mp4  
└─ main.py  
└─ requirements.txt  
└─ chat_history.pdf
```

So your final checklist is:

- ☒ Task 1 completed
- ☒ Task 2 completed
- ☒ Task 3 completed
- ☒ Task 4 completed
- ☒ Git commits made separately as required
- ☒ Pushed to **your fork**, not the original repo
- ☐ Put **before video** in `videos/`
- ☐ Put **after video** in `videos/`
- ☐ Export this chat as **PDF/DOC**
- ☐ Put chat history into the Lab-4 submission folder
- ☐ Final check before submission

So yes: you do **NOT** need to put the chat screenshots individually. The instruction specifically says “Chat history exported as a doc/pdf”, so one exported conversation document/PDF is what you need.



Chat history exported as a doc/pdf”, so one exported conversation document/PDF is what you need.
How to do this

Yep. The easiest way is to export this ChatGPT conversation as a PDF.

Option 1 — Print the conversation to PDF

On your computer:

1. Open this conversation.
2. Press `Ctrl + P`.
3. In the printer/destination options, choose `Save as PDF`.
4. Click **Save**.
5. Name it something like:

`Chat_History_Lab4_Breakout.pdf`

6. Save it inside your project:

```
Lab4_10_breakout/  
└─ breakout/  
    ├── game/  
    ├── videos/  
    │   ├── before.mp4  
    │   └─ after.mp4  
    ├── main.py  
    ├── requirements.txt  
    └─ Chat_History_Lab4_Breakout.pdf
```

Important

Before printing, **scroll through the conversation / make sure the relevant chat history is loaded**, because the PDF will only contain what the browser has rendered/loaded.

You don't need to manually screenshot every message.

Then verify

From your `breakout` folder:

PowerShell

```
dir
```

You should see:

```
game  
videos  
main.py  
requirements.txt  
Chat_History_Lab4_Breakout.pdf
```

Then add the PDF to Git if the lab expects the whole folder/repo to contain the deliverables:

PowerShell

```
git add Chat_History_Lab4_Breakout.pdf videos  
git commit -m "Add Lab 4 videos and chat history"  
git push origin main
```

One thing I'd do before the final commit: make sure your `videos` folder actually contains both the **before** and **after** recordings, with clear filenames.



ChatGPT can make mistakes. Check important info.